



Security scanning, tested.

Real agent and MCP projects. Reproducible results. Explicit limits.

Benchmark execution: Invarune 0.8.0 | 19 September 2026
Navigation and review-workflow edition updated for Invarune 0.9.0. The pinned benchmark measurements remain the recorded 0.8.0 runs.

PUBLIC PROJECTS	INVARUNE PATTERNS	COVERAGE GAPS
8	149	15

What this evaluation establishes

We actually ran Invarune, Semgrep CE, Bandit and Gitleaks against the same selected source bytes from eight public projects at pinned commits. Cisco MCP Scanner ran a separate offline YARA test on 14 literal tool descriptions. No target application or MCP server was executed.

The real-project counts are a review workload, not confirmed vulnerabilities or precision/recall estimates. Manual inspection found false positives and deployment-dependent capabilities. Coverage failures stay visible.

A separate ten-case development fixture comparison exercises five shared API-level patterns with positive and negative examples. It is intentionally small and cannot identify a universal winner.

[Repository](#), [detailed reports](#) and [reproduction scripts](#)

Contents

Click a contents entry or use the PDF bookmarks. Counts describe observed work and coverage, not percent secure or scanner superiority.

Security scanning, tested.	1
01 / Observed findings	3
Measured coverage	4
02 / What each tool tested	5
03 / Review findings in context	6
04 / Shared-pattern checks	7
Configuration and review workflow	8
05 / Reproduce and inspect	9

01 / Observed findings

The same source export was offered to every tool. Supported languages, enabled rules and definitions of a finding differ. A larger or smaller count is not a quality ranking.

Project	Invarune	Semgrep	Bandit	Gitleaks
mcp-reference	61	0	0	0
github-mcp	15	1 *	N/A	0
autogen	13	6	183	0
crewai	28 *	8 *	232 *	0
langgraph	23 *	8	84	1
openhands	9 *	0 *	N/A	1
pydantic-ai	0 *	2	344	0
fastmcp	0	2	95	1

* Analysis gaps or parser errors are present. N/A means no supported Python files for Bandit; it is not a clean result. Source-only scans do not establish deployed behavior, full framework coverage or exploitability.

Scope and timing

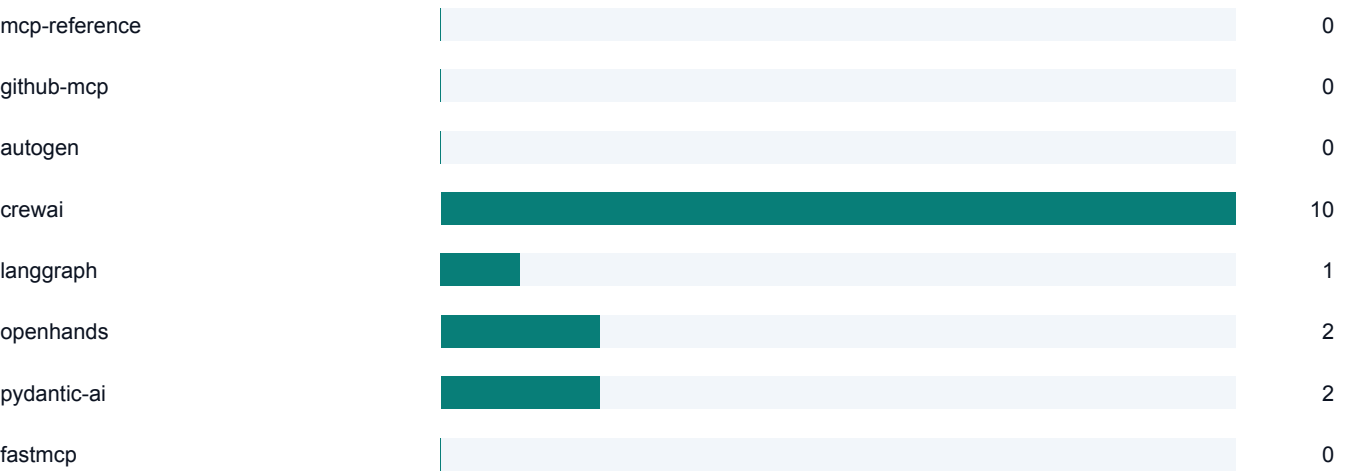
First-party implementation and package/runtime configuration were selected before results were inspected. Tests, docs, examples, vendored/generated directories and non-code assets were excluded uniformly. Each receipt records exact file hashes, options, elapsed time, tool exit status and analysis scope. Timings are single-host observations, not a controlled performance benchmark.

[Pinned corpus, project scope and licenses](#)

[Full per-project reports: HTML, Markdown, JSON and SARIF](#)

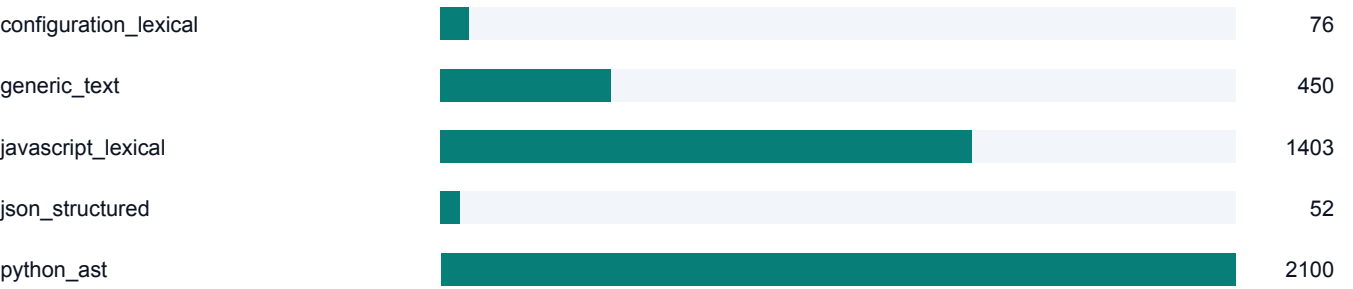
Measured coverage

Coverage failures were retained rather than reclassifying incomplete scans as clean. Four projects have at least one gap. Source template files and large bounded-analysis cases remain in the frozen scope.



Recorded Invarune gaps; counts are not an accuracy or risk score.

Files by assigned analysis profile



Profile assignment does not mean complete analysis; the reported gaps still apply. Go/C# and other generic-text inputs do not receive Invarune's Python local-flow analysis. Tool coverage is not interchangeable even when the exported input bytes match.

02 / What each tool tested

Tool / version	Executed scope	Interpretation
Invarune 0.8.0	42 source/configuration rules; control mapping and coverage evidence	Selected static signals. Whole-program behavior and runtime controls remain open.
Semgrep CE 1.177.0	Frozen p/security-audit pack; 225 rules; metrics off	CE and this pack only. Not Semgrep's commercial engine or a complete rule portfolio.
Bandit 1.9.4	All packaged Python plugins; no severity/confidence filter	Includes capability/import warnings; different finding granularity from Invarune.
Gitleaks 8.30.1	Packaged secret rules; selected source tree; fully redacted	No Git-history scan, credential validity check or general SAST.
Cisco MCP Scanner 4.8.4	YARA only on 14 literal filesystem-tool descriptions	0 matches; partial metadata only. No real tools/list capture, behavioral or LLM analysis.
Snyk Agent Scan	Researched; not executed	Its normal MCP discovery starts configured servers and uses a cloud analysis token. Not a source-only equivalent.

The shortlist covers complementary source analysis, secret detection and MCP metadata inspection. It is a researched, practical sample of established tools, not an exhaustive market survey or an independently adjudicated 'best scanner' award.

Tool provenance, exact rule-pack hash, methodology and restrictions

03 / Review findings in context

The corpus was visible during development. These observations led to narrow detector corrections before the final scan; the original findings and scanner fingerprints remain in the initial triage record.

Observed pattern	Manual interpretation / response
CrewAI PEM marker in an API docstring	False positive: marker followed by an ellipsis, no key body. The detector now requires plausible encoded material.
Pydantic AI dummy SDK API keys	False positives: exact documented placeholder sentinels. Those sentinels are excluded; real credentials remain in scope.
OpenHands Dockerfile USER root	Build setup later switches to a nonroot runtime user. Final-stage USER tracking removes this misleading runtime signal.
LangGraph composed SQL	One reviewed call uses parameterized values and a fixed-predicate helper. Local composition detection cannot establish that whole call chain; review remains required.
AutoGen dynamic Python config loading	An intentional execution capability with an upstream trusted-config warning. It requires a provenance/authorization boundary; no external exploit was demonstrated.
CrewAI environment/provider labels	Credential-named dictionary keys can map to display labels. This context-sensitive false-positive class remains documented.

Immediate review priorities

Validate actual input provenance for dynamic execution and deserialization. Review secret-like values without disclosing them. Resolve parser/work-budget gaps before relying on clean subsets. Check deployed authorization, tool permissions and network exposure independently.

Mitigating layers include isolated execution, least-privilege credentials, per-tool authorization, constrained egress, approval for sensitive actions and monitoring. These are proposed defenses, not observed protections or automatic severity reductions.

[Pinned source links and original finding evidence](#)

04 / Shared-pattern checks

Ten existing, project-authored development fixtures: eval, subprocess shell=True, unsafe YAML loading, pickle loading and disabled TLS verification, each paired with a negative example. Only the explicitly mapped rule is scored; unrelated import/capability warnings do not become false positives for that assertion.

Tool	True positive	True negative	False positive	False negative
invarune	5	5	0	0
semgrep	5	5	0	0
bandit	5	5	0	0

These cases demonstrate bounded compatibility on five selected API patterns. They do not establish production accuracy, broad language coverage, resistance to adversarial code or agent prompt-injection behavior. Gitleaks, Cisco metadata YARA and unexecuted tools receive no artificial true negatives.

Separate Invarune accuracy corpus

The repository also publishes a larger 109-assertion source-pattern corpus with unresolved false positives and false negatives. A versioned PEM fixture update accompanies the narrower key-material rule; the original corpus and results are retained so the change is auditable.

[Shared cases, rule mappings and every result](#)

[Larger Invarune accuracy report and known limitations](#)

Optional model review

Deterministic scanning remains the default. Optional API/gateway or official CLI review adds advisory assessments only. Invarune selects Astra, Opus and Grok Build defaults, validates IDs and exact evidence quotes, and keeps model advice separate from findings and CI gates. Interactive login happens through the official CLI and resumes within the scan.

[Dated live CLI validation receipts and limitations](#)

Configuration and review workflow

This workflow documents current Invarune 0.9.0 behavior. It does not change or rerun the dated 0.8.0 benchmark measurements.

Step	Explicit choice and evidence boundary
Choose target	Source directory or built-image reference/archive. No target application or container is started.
Bound the scan	Set limits and exclusions; retain parser, unsupported-input and work-budget gaps.
Choose optional review	Use an API/custom gateway or official Codex/Claude/Grok CLI only when enabled. Set finding/check/evidence/call/time budgets; omissions remain explicit.
Record human decisions	Edit report review fields with decision, reason, reviewer, date and supporting evidence reference. Justified/disabled are exceptions, not passes.
Rescan and import	Use --review-report with a fresh explicit source/image target. Verify origin and evidence bindings; preserve stale decisions for audit.
Resolve remaining work	Runtime/human-validation requests remain outstanding. Coverage gaps cannot be waived. Failed PDF field/appearance validation falls back to editable JSON, not a clean result.

Repeatable patterns are only one layer

Static checks can miss cross-file flow, reflection, generated code and deployment overrides. Optional model review can add context but can also omit or misinterpret evidence. Neither layer validates deployed OAuth, tenant boundaries, real egress, sandbox escapes or adaptive prompt injection without a separate authorized runtime experiment.

- Method coverage, configured limits and review import CLI
- Source-linked controlbook and human/runtime validation requirements

05 / Reproduce and inspect

Every project is pinned to a full commit in the manifest. The exporter reads Git blobs without checking out or executing target scripts; it verifies the exported file manifest before scans. Reruns reject modified or unexpected files. Detailed source licenses and exclusions accompany each project.

[mcp-reference / d73f99efbfd40c3aa1b61e88728b3d49fb52608f](#)

[github-mcp / 85598ba6e1256f7ebf4867b95d63b833c4549264](#)

[autogen / 027ecf0a379bcc1d09956d46d12d44a3ad9cee14](#)

[crewai / 3831e8b6c86f78cb3cde18ebf7be0d197b958f0e](#)

[langgraph / aa742fb31e2827d569b843e3600aeda2e0528e4b](#)

[openhands / a07364828c8f202e7745c6bce3dcef3915ae7ac1](#)

[pydantic-ai / c4898abb54dc25ae6f6aef208a4c0661b30a455e](#)

[fastmcp / 9c35c017cd89e4d50a9f512c8eafde68c301ec70](#)

Reproduction entry points

Use `scripts/scan_public_projects.py --help` for explicit `fetch/export/scan/repeat` options. Use `scripts/benchmark_competitors.py --help` with separately installed pinned tools and the verified Semgrep rule pack. Exact commands, tool distributions, source hashes and normalized output are published; private raw logs are excluded.

The Semgrep rule pack is not redistributed. Its recorded URL and SHA-256 identify the artifact used; if the upstream pack changes, do not silently replace it and call the run identical.

Scanner implementation SHA-256: `bb306fbd1b72f02daa098336512dc53e87c54d6bbb5aa31a6373967df7db483`

Corpus manifest SHA-256: `8f0115f7f24d81a15dfbb9c67e555bd33f652f129b73b19277419a346e2a096e`

[Source runner and complete reproducibility instructions](#)

[External-tool reproduction commands and primary sources](#)

[Controlbook: 66 controls, 132 checks, 75 primary references](#)

Publication scope

This report is published in the existing NimeshBuild repository. It does not certify the scanned projects, establish an exploit, claim endorsement by a vendor or standards body, or assert universal scanner/model superiority.